

代码清单8给出多中继连续检查的主要入口。程序先组织所有可能改变可用性的时间边界，再对每个区间调用保守认证；无法直接证明的区间继续递归细分，最终汇总最小下界余量和未认证区间。

Listing 8 多中继连续区间认证入口

```

1757 def adaptive_multi_relay_check(
1758     dem: DemGrid,
1759     depot: Node,
1760     phases: list[dict[str, Any]],
1761     relay_points: list[tuple[float, float, float]],
1762     comm: dict[str, float],
1763     thresholds: dict[str, float],
1764     tol_s: float = 0.1,
1765     grid_step_s: float = 0.5,
1766 ) -> dict[str, Any]:
1767     """Continuous combined-link proof for one route and zero or more active relays."""
1768
1769     gateway = (depot.lon, depot.lat, depot.ground_m + comm["天线离地高度 (m)"])
1770     freq, obs = comm["载波频率 (MHz)"], comm["地形遮挡附加损耗 (dB)"]
1771     backhaul = [
1772         interval_link_certificate(dem, point, point, gateway, thresholds["relay_gateway"], freq, obs)
1773         for point in relay_points
1774     ]
1775     if any(not x["certified_available"] for x in backhaul):
1776         raise RuntimeError("Selected full-run relay point lacks a conservative backhaul certificate")
1777
1778     def certify(t0: float, t1: float) -> tuple[bool, float, str]:
1779         p0, p1 = position_at(phases, t0), position_at(phases, t1)
1780         direct = interval_link_certificate(
1781             dem, p0, p1, gateway, thresholds["transport_gateway"], freq, obs
1782         )
1783         if direct["certified_available"]:
1784             return True, float(direct["margin_lower_db"]), "direct"
1785         feasible: list[tuple[float, str]] = []
1786         for index, (point, back) in enumerate(zip(relay_points, backhaul)):
1787             access = interval_link_certificate(
1788                 dem, p0, p1, point, thresholds["transport_relay"], freq, obs
1789             )
1790             margin = min(float(access["margin_lower_db"]), float(back["margin_lower_db"]))
1791             if access["certified_available"]:
1792                 feasible.append((margin, f"relay{index}"))
1793         if feasible:
1794             margin, mode = max(feasible)
1795             return True, margin, mode
1796         return False, float(direct["margin_lower_db"]), "unresolved"
1797
1798     leaves: list[dict[str, Any]] = []
1799     for phase in phases:
1800         phase_plan_m = horizontal_distance_m(*phase["p0"][:2], *phase["p1"][:2])
1801         initial_parts = max(1, int(math.ceil(phase_plan_m / 300.0)))
1802         cuts = np.linspace(float(phase["t0"]), float(phase["t1"]), initial_parts + 1)
1803         stack = [(float(cuts[i]), float(cuts[i + 1])) for i in range(initial_parts)]
1804         while stack:
1805             t0, t1 = stack.pop()
1806             available, margin, mode = certify(t0, t1)
1807             if available:
1808                 leaves.append({"t0": t0, "t1": t1, "mode": mode, "margin_lower_db": margin})
1809             elif t1 - t0 <= tol_s + 1e-12:
1810                 leaves.append({"t0": t0, "t1": t1, "mode": "unresolved", "margin_lower_db": margin})
1811             else:
1812                 tm = (t0 + t1) / 2.0
1813                 stack.extend([(tm, t1), (t0, tm)])
1814         leaves.sort(key=lambda x: (x["t0"], x["t1"]))
1815     segments: list[dict[str, Any]] = []
1816     for leaf in leaves:
1817         if segments and segments[-1]["mode"] == leaf["mode"] and abs(segments[-1]["t1"] - leaf["t0"]) <= 1e-7:
1818             segments[-1]["t1"] = leaf["t1"]
1819             segments[-1]["margin_lower_db"] = min(
1820                 segments[-1]["margin_lower_db"], leaf["margin_lower_db"]
1821             )
1822         else:
1823             segments.append(dict(leaf))
1824
1825     grid_times = np.arange(phases[0]["t0"], phases[-1]["t1"] + 1e-12, grid_step_s)
1826     if grid_times[-1] < phases[-1]["t1"]:

```